

Advanced SQL Assignment for MySQL

This assignment is designed to test your understanding and application of advanced SQL concepts in MySQL. You are expected to demonstrate your proficiency in constructing complex queries using `SELECT`, `WHERE`, `ORDER BY`, `IN`, `LIKE`, comparison operators, logical operators, `GROUP BY`, `HAVING`, subqueries, and Common Table Expressions (CTEs).

Consider the following tables for a fictional online bookstore. For this assignment, assume you will primarily be querying individual tables or using subqueries to relate data without explicit `JOINS`.

Books

Column Name	Data Type	Description
book_id	INT (Primary Key)	Unique identifier for each book
title	VARCHAR(255)	Title of the book
author_id	INT	Foreign key referencing Authors table (for conceptual understanding, but will be used in subqueries only)
genre	VARCHAR(100)	Genre of the book
price	DECIMAL(10, 2)	Price of the book
publication_year	INT	Year the book was published
stock_quantity	INT	Number of copies available in stock

Authors

Column Name	Data Type	Description
author_id	INT (Primary Key)	Unique identifier for each author
first_name	VARCHAR(100)	Author's first name
last_name	VARCHAR(100)	Author's last name

nationality	VARCHAR(100)	Author's nationality
-------------	--------------	----------------------

Orders

Column Name	Data Type	Description
order_id	INT (Primary Key)	Unique identifier for each order
customer_id	INT	Foreign key referencing Customers table (for conceptual understanding, but will be used in subqueries only)
order_date	DATE	Date the order was placed
total_amount	DECIMAL(10, 2)	Total amount of the order

Order_Items

Column Name	Data Type	Description
order_item_id	INT (Primary Key)	Unique identifier for each order item
order_id	INT	Foreign key referencing Orders table (for conceptual understanding, but will be used in subqueries only)
book_id	INT	Foreign key referencing Books table (for conceptual understanding, but will be used in subqueries only)
quantity	INT	Quantity of the book in the order item
item_price	DECIMAL(10, 2)	Price of the book at the time of purchase

Customers

Column Name	Data Type	Description
customer_id	INT (Primary Key)	Unique identifier for each customer
first_name	VARCHAR(100)	Customer's first name

last_name	VARCHAR(100)	Customer's last name
email	VARCHAR(255)	Customer's email address
registration_date	DATE	Date the customer registered

Assignment Questions

Please write MySQL queries for each of the following questions, ensuring no explicit JOIN statements are used. Focus on using subqueries (IN, EXISTS, NOT EXISTS), aggregate functions, and other non-JOIN methods.

- List the titles and prices of all books published before 2000 that cost more than 25.00, ordered by price in descending order.
- Retrieve the `book_id`, `title`, and `stock_quantity` for books that have a `stock_quantity` less than the average `stock_quantity` of all books in the 'Mystery' genre.
- Show the `customer_id` and `email` of customers who have placed at least 3 orders (count from the `Orders` table) and whose email address contains 'example.com'.
- Find the `book_id` and `title` of books that have been ordered more than 5 times in total across all `Order_Items` records, but whose `price` (from the `Books` table) is less than 30.00.
- Get the `title`, `author_id`, and `price` of books that have a price higher than any book written by author with `author_id = 101`.
- Find the `order_id` for orders that contain more than one unique `book_id` in their `Order_Items`.
- List the titles of books that have a 'History' genre and were published in the same `publication_year` as any book with `book_id` 105.
- Display the `book_id`, `title`, and `genre` of books that have a `stock_quantity` below the average `stock_quantity` across all books, and are in either 'Fiction' or 'Biography' genres.
- For each `book_id`, calculate the total `quantity` sold across all `Order_Items` where the `item_price` was greater than 15.00. Only display `book_ids` where the total quantity sold is greater than 10.
- Identify `customer_ids` from the `Customers` table who have an email address containing 'gmail.com' and have placed at least one order.
- List the `order_ids` from the `Orders` table where the `total_amount` is among the top 5 highest `total_amounts` recorded.
- Find the `author_id` and `nationality` of authors who have written books with a `price` between 20.00 and 40.00.

13. Retrieve the `order_id` and `order_date` for orders placed in the last 90 days from the current date.